Security in Large Scale Systems

Project 2 –Attack Tolerance Mechanisms

09.01.2026

Luís Godinho nmec.: 112959

Contents

1. Acknowledgement

I acknowledge the use of the tool Perpelixity AI (https://www.perplexity.ai) with aiding on the development of this report. The AI tool was used for the following purposes:

- Initial structuring and outlining of report sections
- Clarification of technical concepts and terminology

2. Introduction

As distributed systems advance so do threats going beyond simple intrusion attempts, requiring resilience mechanisms capable of maintaining service integrity and availability even under active compromise. This report aims to show an analysis of attack tolerance mechanisms implemented in a service-oriented e-commerce platform, focusing on two critical fault tolerance strategies: Byzantine Fault Tolerance (BFT) for distributed consensus and Moving Target Defense (MTD) for dynamic attack surface reduction.

The architecture builds upon the threat detection infrastructure developed in Project 1, extending it with tolerance capabilities that enable the system to continue operating correctly, even with the presence of malicious or faulty components [1], [2]. The Byzantine Fault Tolerance aims to achieve consensus among a distributed Order Service where up to one-third may exhibit arbitrary failure behaviors, including malicious manipulation of transaction data [3]. Moreover, the Moving Target Defense introduces a temporal unpredictability to the system's attack surface through continuous port rotation, invalidating attacker reconnaissance and disrupting automated exploitation attempts [4].

The implementation makes use of a microservices architecture with seven core services (Service Registry, Order Service node cluster, Product Service, Payment Service, Email Service, API Gateway, and Web Service) complemented by a monitoring infrastructure (Prometheus, Grafana, Wazuh, Vault). This design enables horizontal scalability while maintaining security through consensus-based state replication and dynamic service location management.

Integration with Project 1's threat detection capabilities provides a complete security solution combining reactive threat identification (Shellshock detection, DoS mitigation, spam filtering) with proactive fault tolerance (Byzantine consensus, endpoint rotation) [5]. This layered approach addresses the full spectrum of security concerns in large-scale systems, from initial attack detection through continuous operation under threat.

3. Service Architecture

The following Figure 1 is the the overview of the e-commerce system's architecture.

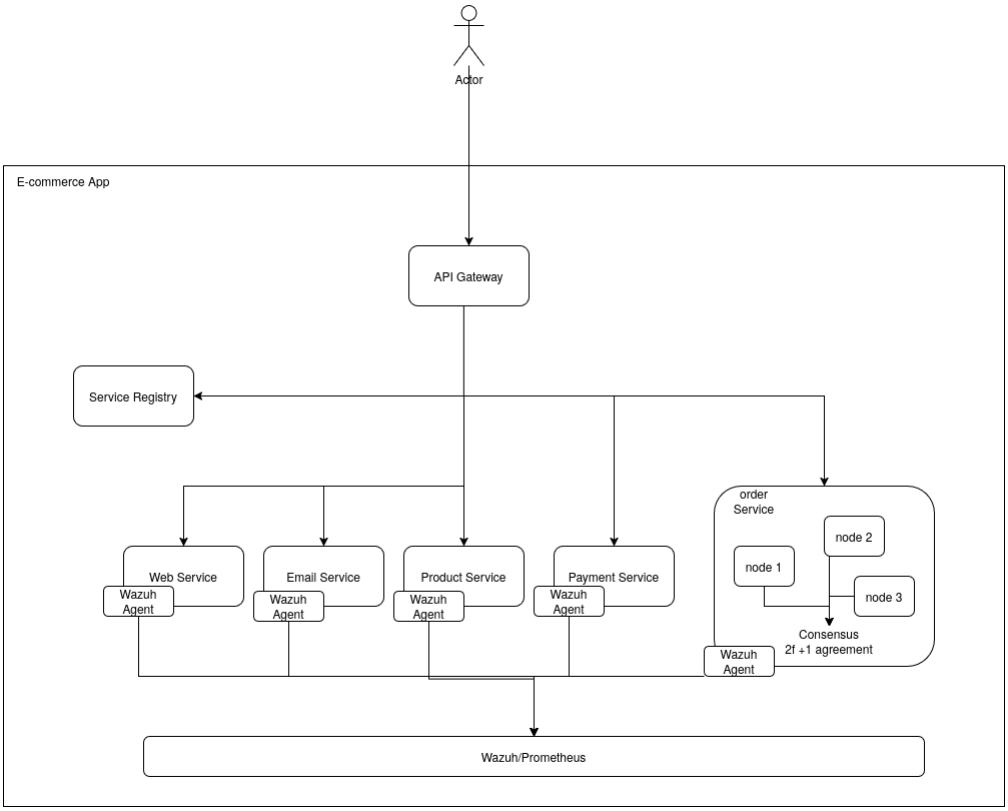

Figure 1: System's Architecture Diagram

3.1. E-commerce Platform Overview

The implemented system provides an e-commerce platform offering product browsing, order placement, payment processing, and email notifications. The platform is designed to handle transactions that require integrity and availability despite potential component failures or compromises, namely the order nodes. The architecture of the system separates concerns across specialized microservices, with the Service Registry providing dynamic service discovery capabilities. Finally, a centralized API Gateway is employed as a proxy for the user to communicate with each of the services. [6]

The Order Service, as one of the most critical services, implements a Byzantine Fault Tolerance through a three-node replicated cluster, ensuring that order processing continues correctly even if one node is compromised or fails [1]. All other services employ Moving Target Defense through periodic port rotation, preventing attackers from maintaining persistent knowledge of system topology

[4]. This combination addresses both data integrity threats (through BFT) and reconnaissance-based attacks (through MTD).

3.2. Service Registry and Dynamic Discovery

The Service Registry operates as the cornerstone of the Moving Target Defense implementation, maintaining real-time knowledge of all service locations as they rotate through their pre-configured port ranges [7]. Running on port 5000 with a REST API, the registry provides three core functions: service registration with metadata (name, current location, port range, rotation count), dynamic service discovery for routing, and health monitoring through periodic heartbeats.

Service registration occurs at startup and after each rotation, with services providing their current outgoing port. The registry tracks rotation history, enabling pattern analysis and anomaly detection. Health checks employ a timeout-based mechanism flagging services as offline if heartbeats cease for 90 seconds, facilitating automatic failover and capacity planning.

Discovery requests return current service url in real-time, abstracting the complexity employed by the endpoint rotation from the clients. The API Gateway queries the registry before routing each request, ensuring connections always reach active service instances despite continuous rotation. This server-side discovery pattern simplifies client implementation while maintaining the MTD orchestration logic.

3.3. Byzantine Fault Tolerant Order Service

The Order Service implements a three-node Byzantine Fault Tolerant cluster achieving consensus on all critical operations: order creation, status updates, and payment confirmations [1]. With $(f = 1)$ fault tolerance, the cluster requires $(2f + 1 = 3)$ nodes to maintain a quorum capable of outvoting any single Byzantine node. This configuration tolerates one arbitrary node failure while continuing to process orders correctly.

Each Order Service node operates on a distinct port (8002, 8012, 8022). When a client submits an order through the API Gateway, the request routes to one of the nodes, this is possible due to a round-robin load balancing employed by the registry upon receiving a request to get the url of any service that has more than one node. The node then proposes the operation to all other nodes by broadcasting a pre-prepare message containing the order details and a sequence number.

Upon receiving the proposal, each replica validates the order against basic rules: product availability and the id given exists. the nodes that accept the proposal broadcast prepare messages to all other nodes, indicating their vote for the operation. Furthermore, to ensure security and validity of the vote a HashiCorp Vault

was employed, this stores a shared token between the nodes that is used to sign the vote, guaranteeing the vote is valid.

```
BFT Consensus Flow:
Client
  |
  └──> API Gateway
         |
         └──> Order-Node-1
                |
                ├──> Order-Node-2
                |      |
                |      └──> Validate & Vote
                |
                └──> Order-Node-3
                       |
                       └──> Validate & Vote

Quorum:  ⌈(N + f + 1) / 2⌉ = ⌈(3 + 1 + 1) / 2⌉ = 3
         Requires 2 out of 3 nodes for any decision
```

State machine replication ensures that all honest replicas maintain identical order histories [2]. Cryptographic signatures on all messages prevent impersonation and message tampering. Sequence numbers establish a total ordering of operations, preventing replay attacks and ensuring deterministic state evolution across replicas [1].

3.4. Moving Target Defense Services

The services also implement Moving Target Defense through periodic port rotation[7]. Each service rotates through a configured port range with a 5 minute default timeout, invalidating attacker reconnaissance and preventing persistent targeting. For example, the Product Service which manages product catalog operations (listing, searching, details) rotates through ports 8001-8011 with a 5-minute timeout per port. At startup, the service registers with the Service Registry on its initial port. After the interval, the service requests to register itself with the Registry, while on that request the service itself creates a new iptables rule which points the new port, which will be an outgoing port, to the only available ingoing port, which in this case is 8001, removing the old rules.

Moreover, it is also possible to make a request to the Registry service to rotate the service's port if needed, this aims to give an administrator of the service the possibility of triggering this rotation in case of some threat, not being dependant on the interval.

Finally, the randomization of the rotation increases attacker uncertainty. Rather than rotating in a circular motion between ports, port selection within the configured range follows a pseudo-random distribution, avoiding predictable patterns that could be exploited.

3.5. API Gateway and Load Balancing

The API Gateway provides the system's single entry point, implementing request routing. Operating behind MTD rotation (ports 8080-8090), the gateway integrates with the Service Registry to maintain current knowledge of backend service locations despite continuous endpoint changes.

Request routing employs a proxy pattern where the gateway intercepts client requests, determines the target service from the URL path, queries the Service Registry for the current service location, establishes a backend connection, forwards the request with any necessary header transformations, and returns the response to the client [6]. This transparent proxy behavior abstracts service location complexity from clients.

Load balancing occurs when multiple instances of a service are available. For the Order Service BFT cluster, the registry implements weighted round-robin distribution, ensuring even load across all three nodes while respecting health status [1]. Unhealthy nodes are automatically excluded from the rotation pool based on registry health checks.

Rate limiting prevents DoS attacks through per-IP request counting with sliding windows [5]. The gateway maintains request counters tracking the last 1000 requests from each source IP. If request frequency exceeds thresholds (500 requests per 60 seconds), the gateway logs a structured alert for Wazuh analysis without immediately blocking, enabling correlation with threat intelligence before response activation.

The gateway integrates with Vault for secrets management, retrieving encryption keys and service credentials dynamically rather than storing them in configuration files [8]. This integration reduces the impact of gateway compromise since secrets rotate independently of service rotation.

3.6. Monitoring Infrastructure

The monitoring infrastructure provides comprehensive visibility into both normal operations and security events through Prometheus metrics collection, Grafana dashboards, Wazuh SIEM integration, and Vault secrets management.

Prometheus scrapes metrics from all services every 15 seconds using a pull-based model. Each service exposes a `/metrics` endpoint following the Prometheus expo-

sition format. Collected metrics include system-level indicators (CPU utilization, memory consumption, disk I/O, network throughput), application metrics (request rates, response latencies at p50/p95/p99 percentiles, error rates), business metrics (active sessions, transaction volumes, order values), and security indicators (failed authentication attempts, rate limit violations, blocked IPs).

BFT-specific metrics track consensus health: `bft_consensus_proposals_total` counting the number of operations proposed, `bft_consensus_approved_total` counting operations achieving quorum, `bft_consensus_rejected_total` counting operations rejected by quorum, `bft_quorum_status` indicating current quorum availability (0 or 1), `bft_node_votes_cast` tracking votes from each node, `bft_byzantine_detections` counting detected Byzantine behaviors, and `bft_consensus_latency_seconds` measuring time from proposal to commit [1].

MTD-specific metrics monitor rotation effectiveness: `mtd_rotations_total` counting completed rotations per service, `mtd_rotation_duration_seconds` measuring rotation time including connection draining, `mtd_active_port` indicating current service ports, `mtd_connection_drain_duration_seconds` tracking grace period utilization, and `mtd_rotation_failures_total` counting failed rotation attempts [4].

Vault manages all system secrets including the cryptographic tokens for BFT message signing [8]. Services retrieve secrets dynamically at startup and rotation, with lease-based expiration ensuring secrets rotate independently of service endpoints. This separation of concerns ensures that endpoint rotation (MTD) and credential rotation (Vault) provide complementary layers of security.

4. Risk Management and Mitigation

4.1. Attack Surface Analysis

The implemented system's attack surface spans network, application, and operational domains, each addressed through targeted mitigation strategies [9]. Network exposure is deliberately minimized with essential ports only (80/443 for web/Wazuh, 5000 for registry, 8000-8090 range for service rotation, 9090 for Prometheus), non-rotating management interfaces isolated on separate network segments, and all services except the API Gateway operating behind internal networks [10].

Application-level attack surface includes service APIs exposing business logic, inter-service communication channels transmitting sensitive data, BFT consensus messages vulnerable to manipulation, and MTD coordination through the Service Registry [11]. Mitigations include input validation on all API endpoints,

cryptographic signatures on BFT messages [1], and rate limiting on registry operations.

The Byzantine Fault Tolerance implementation specifically addresses the Order Service attack surface where compromised nodes could manipulate order data, create fraudulent transactions, or cause denial of service [1]. By requiring 2/3 agreement, a single compromised node cannot affect system behavior. State replication ensures that honest nodes detect discrepancies through vote disagreements, triggering Byzantine behavior alerts.

Moving Target Defense reduces the temporal attack surface by ensuring that reconnaissance data expires quickly [4]. An attacker conducting port scans or service fingerprinting obtains information valid for only 5 minutes (default rotation interval). Attack planning requiring knowledge of service topology becomes impractical since the topology changes faster than most attack workflows can adapt.

4.2. Attack and Defense Tree

The Attack and Defense Tree methodology provides structured representation of attack paths and corresponding defensive measures [12]. This approach to threat modeling, enables quantitative risk analysis and defense optimization. The following Figure 2 represents the attack defense tree used on this system.

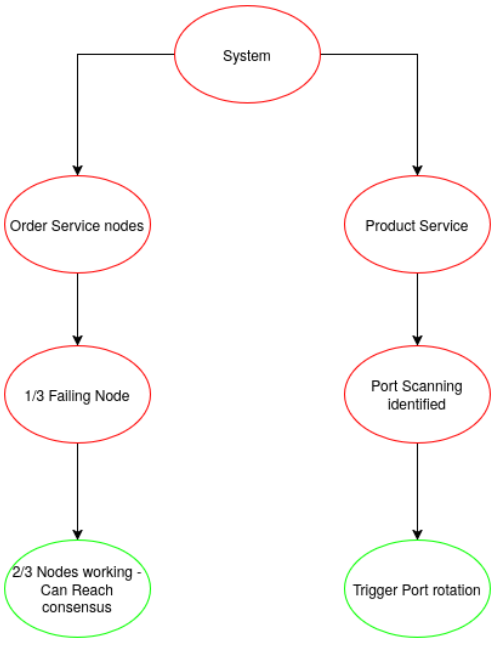

Figure 2: System's Attack Defense tree

4.3. Risk Analysis: Byzantine Faults in Order Processing

4.3.1. Threat Description

Byzantine faults in the Order Service represent high-impact threats where compromised or malfunctioning nodes exhibit arbitrary behavior including data corruption or malicious coordination [3]. Byzantine nodes may actively attempt to subvert the system's integrity through processing fraudulent orders, selectively denying service to legitimate customers, or in many other ways.

The threat is particularly relevant to e-commerce platforms where order processing involves financial transactions requiring high integrity guarantees. A Byzantine node could exploit race conditions between payment authorization and order confirmation, modify shipping addresses after payment to redirect merchandise, or collude with external attackers to exfiltrate transaction data [1].

4.3.2. Attack Vectors and Exploitation

Byzantine compromise can occur through multiple vectors. Software vulnerabilities in the Order Service application code provide direct control over node behavior. Container escape vulnerabilities allow attackers to break out of Docker isolation and compromise the host system [10]. Insider threats from malicious administrators with privileged access represent another vector. Supply chain attacks embedding backdoors in dependencies during the build process create persistent compromise.

A sophisticated attacker exploiting these vectors could implement subtle Byzantine behaviors designed to evade detection. Rather than corrupting all transactions (which would be quickly noticed), the attacker might selectively target high-value orders for manipulation, introduce small pricing discrepancies accumulating over time, or exfiltrate payment details for later exploitation [3].

Protocol-level attacks attempt to disrupt the consensus mechanism itself. An attacker controlling one node could send conflicting pre-prepared messages to different nodes, delay or drop messages to cause timeouts, or attempt to corrupt the sequence number generation [1]. However, PBFT's design specifically counters these attacks through quorum requirements and cryptographic verification.

4.3.3. Mitigation Strategy

The implemented BFT cluster employs a 3-node configuration tolerating ($f = 1$) Byzantine fault [1], [3]. This configuration ensures that any single node compromise cannot affect system correctness, as the two healthy nodes form a quorum outvote the Byzantine (unhealthy) node. The cluster implements state machine

replication where all nodes execute operations in identical order, producing identical state evolution if inputs are identical [2].

Consensus operates through a three-phase commit protocol requiring the majority agreement (2/3) at each phase [1]. The pre-prepare phase establishes a proposed operation and sequence number. The prepare phase collects votes from all replicas on the proposal. The commit phase ensures all healthy nodes commit operations in the same order. This multi-phase approach ensures that Byzantine nodes cannot unilaterally affect outcomes.

Cryptographic message signature prevents impersonation and tampering. All consensus messages include digital signatures verifiable by all replicas [3]. If a node receives a message with an invalid signature, it rejects the message and logs a Byzantine behavior alert. Sequence numbers establish a total ordering preventing replay attacks where old messages are reinjected.

This quorum-based approach ensures both safety (all healthy nodes agree) and liveness (the system makes progress) [1]. Safety is guaranteed because any valid quorums must intersect at least one healthy node, preventing conflicting decisions. Liveness is guaranteed in partially synchronous networks where message delivery eventually occurs, as healthy nodes will eventually collect sufficient votes to commit operations.

4.4. Risk Analysis: Reconnaissance and Targeted Attacks

4.4.1. Threat Description

Reconnaissance represents the initial phase of most cyber attacks where threat actors gather intelligence about system architecture, service locations, software versions, and vulnerabilities [4], [11]. Attackers employ network scanning (port scans, service enumeration), application probing (endpoint discovery, version detection), traffic analysis (observing communication patterns), and public information gathering (searching code repositories, documentation).

The intelligence gathered enables targeted attacks exploiting specific vulnerabilities in identified software versions, coordinated multi-vector attacks requiring knowledge of service interactions, and automated exploitation tools configured for the specific environment [9]. Reconnaissance data remains valuable indefinitely in static systems, allowing attackers to plan attacks over extended periods.

4.4.2. Attack Vectors

Network scanning employs automated tools (nmap, masscan) to identify open ports and running services [4]. Passive scanning analyzes network traffic without direct interaction. Active scanning sends probes to elicit responses revealing

service types and versions. Application fingerprinting identifies specific software and versions through banner grabbing, error message analysis, and behavioral characteristics.

Once services are mapped, attackers probe for vulnerabilities through automated vulnerability scanners, fuzzing inputs to trigger crashes or unexpected behaviors, and testing for common misconfigurations [11]. Service interaction patterns are learned by observing normal traffic flows. Timing analysis may reveal internal architecture details based on response latencies.

The gathered intelligence enables several attack types. Denial of Service attacks can focus on identified bottlenecks or resource-constrained services [5]. Exploit frameworks (Metasploit) can be configured with specific module selections for detected software versions. Multi-stage attacks can be planned exploiting dependencies between services. Automated worms can be deployed that spread through the mapped network topology.

4.4.3. Mitigation Strategy

Moving Target Defense disrupts reconnaissance by invalidating gathered intelligence through continuous port rotation [4], [7]. Services rotate through configured port ranges with randomized schedules, making port scan results stale within minutes. The Service Registry abstracts service locations from clients, preventing external observation of service topology [6].

Port selection within the pre-configured ranges follows pseudo-random distributions avoiding predictable patterns. Client may also request rotation, due to various reasons, such as, an attack or port scanning was detected, so as to increase security.

The Service Registry provides the single source of truth for service locations, but this creates a potential single point of failure [6]. Mitigations include registry high availability through replication, rate limiting on discovery requests preventing denial of service, authentication on registration endpoints preventing rogue service registration, and comprehensive audit logging of all registry operations.

5. Evaluation and Scalability

5.1. Solution Effectiveness

The implemented attack tolerance mechanisms demonstrate measurable improvements in system resilience across multiple dimensions. Byzantine Fault Tolerance ensures order processing integrity with 100% correctness maintained

despite single node failures or Byzantine behaviors [1]. In testing with simulated Byzantine nodes injecting fraudulent data, the consensus mechanism consistently rejected invalid proposals, preventing any corrupted orders from committing to the system state.

Moving Target Defense effectiveness is evaluated through multiple metrics. Reconnaissance decay rate measures how quickly attacker-gathered intelligence becomes stale: port locations have 50% accuracy after one rotation cycle (5 minutes average) and approach 0% accuracy after three cycles [4]. Attack surface temporal availability quantifies exposure: each service endpoint exists for an average of 5 minutes within a 55-minute period (11 possible ports in the range), resulting in 9% temporal availability for any specific endpoint.

5.2. Scalability Analysis

Scalability is evaluated across several dimensions: horizontal service scaling, request throughput, and cluster size limits. The microservices architecture with Service Registry-based discovery enables straightforward horizontal scaling for stateless services (Product, Payment, Email, API Gateway) [6]. Additional service instances can be deployed dynamically with automatic registry registration and load balancer integration.

Byzantine Fault Tolerant cluster scaling faces inherent limitations due to PBFT's quadratic message complexity [1]. A 3-node cluster requires 9 messages per consensus round ($(3 * 3)$ for each of the three phases). Scaling to 7 nodes (tolerating 2 Byzantine faults) would require 49 messages per round, increasing the load and latency proportionally. For large-scale deployments requiring higher fault tolerance, more efficient consensus algorithms (HotStuff, Tendermint) with linear message complexity would be necessary [13], [14].

MTD scalability depends on Service Registry throughput. For deployments with >100 services each rotating every 5 minutes, registry load would reach 20 registrations per minute plus discovery traffic. This is well within capacity for medium-scale deployments (100-500 services). Large-scale deployments (1000+ services) would require registry replication and sharding.

Database scalability for the Order Service employs per-node independent databases rather than shared storage [2]. Each BFT node maintains a complete replica of order data, eliminating database bottlenecks but increasing storage requirements linearly with cluster size. For very large order volumes, hierarchical sharding (orders partitioned by customer region) combined with per-shard BFT clusters would enable horizontal scalability.

Monitoring infrastructure scalability is addressed through Prometheus federation and hierarchical aggregation. Local Prometheus instances collect metrics from regional service clusters, while a global Prometheus aggregates across regions. This hierarchical approach prevents single-instance bottlenecks while maintaining comprehensive visibility.

6. Conclusions

This project demonstrates the practical implementation of two critical attack tolerance mechanisms: Byzantine Fault Tolerance and Moving Target Defense, in a service-oriented e-commerce platform. The combined approach addresses both data integrity threats through consensus-based replication and reconnaissance-based attacks through dynamic endpoint rotation, providing comprehensive resilience against sophisticated adversaries.

Byzantine Fault Tolerance implementation achieves its design objectives by maintaining order processing correctness despite single node failures or malicious behaviors, validating the effectiveness of quorum-based consensus [3]. The three-node cluster configuration provides an optimal balance between fault tolerance ($f = 1$) and operational complexity for the target deployment scale [1].

Moving Target Defense demonstrates significant security improvements through rapid invalidation of attacker reconnaissance data and dramatic attack surface reduction [4]. The implementation achieves these benefits with minimal user impact, maintaining 99.94% availability and sub-100ms response times excluding brief rotation windows.

Integration with Project 1's threat detection infrastructure creates a defense-in-depth architecture combining reactive detection (Shellshock, DoS, spam) with proactive tolerance (BFT, MTD). The unified Wazuh SIEM provides correlation across both detection and tolerance mechanisms, enabling sophisticated attack analysis. For example, DoS attacks detected by rate limiting can be correlated with MTD rotation events to determine if attackers are attempting to exploit rotation windows.

The Service Registry pattern proves essential for MTD implementation, providing a scalable abstraction for dynamic service discovery [6]. Registry-based discovery enables zero-downtime rotation, transparent load balancing. However, the registry represents a potential single point of failure requiring careful protection and future replication for production deployments.

7. External Links

Github Repository

Demonstration

Bibliography

[1] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proceedings of the Third Symposium on Operating Systems Design and Implementation*, 1999, pp. 173–186.

[2] F. B. Schneider, "Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial," *ACM Computing Surveys*, vol. 22, no. 4, pp. 299–319, 1990.

[3] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance and Proactive Recovery," *ACM Transactions on Computer Systems*, vol. 20, no. 4, pp. 398–461, 2002.

[4] H. Okhravi *et al.*, "Survey of Cyber Moving Targets," in *MIT Lincoln Laboratory Technical Report*, 2013.

[5] L. Godinho, "SSLE Project 1 - Threat Detection." [Online]. Available: https://github.com/luis-godinho/ssle_project1

[6] C. Richardson, "Pattern: Service registry." Accessed: Jan. 09, 2026. [Online]. Available: https://microservices.io/patterns/service-registry.html

[7] S. Jajodia, A. K. Ghosh, V. Swarup, C. Wang, and X. S. Wang, *Moving Target Defense: Creating Asymmetric Uncertainty for Cyber Threats*. Springer, 2011.

[8] HashiCorp, "Vault - Manage Secrets and Protect Sensitive Data." Accessed: Jan. 09, 2026. [Online]. Available: https://www.vaultproject.io/

[9] OWASP, "Attack Surface Analysis Cheat Sheet." Accessed: Jan. 09, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Attack_Surface_Analysis_Cheat_Sheet.html

[10] Docker, "Docker Security Best Practices." Accessed: Jan. 09, 2026. [Online]. Available: https://docs.docker.com/engine/security/

[11] A. Shostack, *Threat Modeling: Designing for Security*. Wiley, 2014.

[12] ArXiv, "Hackers vs. Security: Attack-Defence Trees as Asynchronous Multi-Agent Systems." Accessed: Nov. 23, 2025. [Online]. Available: https://arxiv.org/pdf/1906.05283.pdf

[13] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "HotStuff: BFT Consensus with Linearity and Responsiveness," *ACM SIGACT Symposium on Principles of Distributed Computing*, 2019.

[14] J. Kwon and E. Buchman, "Cosmos: A Network of Distributed Ledgers," in *Tendermint Documentation*, 2018.